Growing MIDI Music Files Using Convolutional Cellular Automata

Omar Delarosa[1] and L. B. Soros[1]

[1]Tandon School of Engineering, New York University, New York, NY 1120https://www.overleaf.com/project/5e45d151e0c5bc0001c476ec1
omar.delarosa@nyu.edu, lsoros@nyu.edu

Abstract

This paper describes a novel generative system based on cellular automata (CA). The main contribution of this work is *CARLA*, a Cellular Automata Rule Learning Algorithm that learns CA update rules with lower computational overhead (i.e. a smaller neural architecture) than other current approaches. A formal model is developed, situating the algorithm in terms of machine learning. The algorithm is then validated on the 256 possible elementary cellular automata rules. Next, *CARLA* is incorporated into a larger generative system that learns musical patterns by transforming MIDI-encoded music into cellular automata. This system is then used to generate a novel MIDI composition based partially on update rules learned from Chopin's *Waterfall Étude*, demonstrating the utility of this new algorithm for creative applications.

Introduction

The proliferation of accessible deep learning technologies has enabled the development of generative systems operating at previously impossible scales. Network-based generative models such as Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs) have proven successful at creating novel images and music (Jing et al., 2019) from existing data. The fundamental algorithmic challenge, then, is encoding regularities from pre-existing artifacts in a computational model that can then later be used for generative purposes. However, the training of deep neural network architectures can incur a heavy environmental cost (Strubell et al., 2019).

In contrast, cellular automata (CA) have long served as computationally efficient models of discrete dynamical systems. Despite their relative simplicity, CA are capable of generating complex patterns with emergent features such as self-replication. In terms of music specifically, two-dimensional CA can compose patterns of high-quality, discrete rhythmic sequences with minimal computational overhead in a local-state-focused generative system (Brown, 2005). Moreover, understanding and creating generative systems has has been an important theme in artificial life research. Though evolutionary systems have received much attention in recent years, understanding the power and limitations of alternative generative methods can reveal novel insights about creative processes in general.

This paper introduces *CARLA*, a Cellular Automata Rule Learning Algorithm that learns CA update rules from a sequence of example states and then incorporates them into a broader system for generating novel MIDI compositions. The next section gives necessary background on cellular automata and unsupervised learning for update rules. Then, the CARLA algorithm is described and validated on an elementary cellular automata. We then demonstrate how musical sequences can be encoded as CAs and explore how learned musical CA update rules can be used to generate novel musical compositions.

Background

This section first briefly reviews the basics of cellular automata, which will be necessary for understanding the music generation system described in this paper, and then focuses on learning CA update rules with deep networks.

Cellular automata are idealized models of complex systems, made up of only a system of discrete cells and a *cell update rule*, or *transition rule*, that determines each cell's successive state given its current state and the state of its neighbors (Neumann and Burks, 1966). Like the bottom-up strategies of living systems such as ant colonies or avian swarms, a cell changes only in response to its local neighborhood, indifferent to the global system state. Figure 1 illustrates an example update rule for an elementary cellular automaton (ECA), while Figure 2 shows how this local rule generates patterns when applied iteratively.

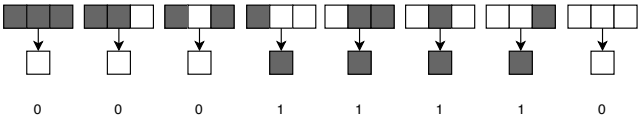


Figure 1: Elementary cellular automata transition rule 30 (Wolfram, 2002).

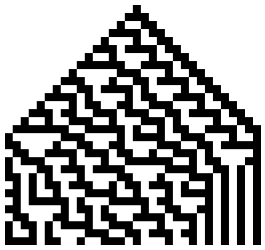


Figure 2: Space-time diagram of ECA rule 30, unfolding from top to bottom. The sole initial "living" cell becomes many as the rule repeats.

Though CAs range in complexity and dimensionality, even simple ECAs have demonstrated emergent complexity in at least two of the four classes of systems they form (Wolfram, 2002). In fact, John von Neumann famously proved that, with the right rule set, CAs are capable of universal computation, hinting at the potential for generating arbitrarily complex patterns (Neumann and Burks, 1966).

Modern CA applications have harnessed deep neural architectures (among other techniques) to encode complex dynamics. For instance, Mordvintsev et al. (2020) demonstrate a system capable of learning CA update rules for "growing" emoji from a single initial cell. More generally, Gilpin (2019) showed that *arbitrary* CAs can be represented by convolutional perceptrons, and specifically that 2D CA update rules can be encoded using a neural network with a single 3x3 convolution layer (Fig. 3) and multi-later perceptron architecture.

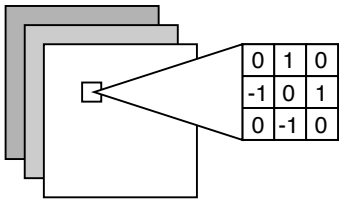


Figure 3: 2-D 3x3 convolution being applied to each pixel on an image

In contrast, the novel *CARLA* system utilizes a shallower, single-layer architecture to achieve unsupervised learning of the cell update rules. Although avoiding the higher-energy-cost deep architectures of CNNs, CARLA maintains the convolutional representation of CA because a cell's locality and neighborhood-first update rules are analogous to convolution kernels used by CNNs. For clarity of concept, the subsequent section uses a 3x1 convolution kernel applied to the 1-D case of elementary CA (or ECAs) as an element-wise operation on an array. However, the results can be generalized to 2-D cases as well using the similar NxN kernel as Gilpin. The value of N varies according to the *kernel radius*, one of the model parameters of *CARLA* that is described in the *CARLA: Architecture* section.

Methodology

This section describes the novel *CARLA* system, which learns CA update rules in an unsupervised manner while incurring relatively low computational overhead.

Formal Model of Convolutional Cellular Automata

For the purpose of connecting the work reported in this paper to existing work in the machine learning community, this section describes cellular automata using notation typically used to describe convolutional neural networks. Cellular automata (CA) can be fully described by three model parameters ϕ, λ and k, respectively their *neighborhood operator*, *activation function* and *kernel radius*. There also exist 2 hyperparameters n and D, respectively describing the number of possible states per cell and the number of dimensions in the state space. The subsequent descriptions focus on the binary, 1-D ECA case and assume $n = 2$ and $D = 1$.

The first model parameter $\phi_{k=r}$ describes the *neighborhood operator* (Fig. 4), or a particular mapping of neighborhood to a new state. The ϕ parameter is akin to a convolution kernel in a CNN but represents an entire transition.

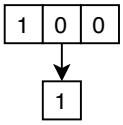


Figure 4: A $\phi_{k=1}$ operator

The second model parameter, the subscript k on ϕ, describes the radius of the neighborhood as r, the number of cells between its outer boundary and its center cell. In Fig. 4, $k = 1$ because there is one cell on each side of the center cell. This parameter is equivalent to the radius of a Moore neighborhood.

The third model parameter is an *activation function* λ describing the cell update rule and the set ϕs. For example, $\lambda_{R_{30}}^{k=1}$ describes Rule 30 (Fig. 5) in $k = 1$ rule space:

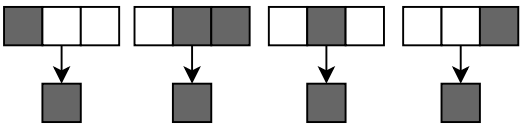


Figure 5: $\lambda_{R_{30}}$ describing the activation function of Rule 30

In the 8-bit ECA, $\lambda_{R_{30}}^{k=1}$ can be described by a single integer 30 representing the binary sequence 00011110 describing the entire rule (Fig. 1) as an ordered set of all ϕ and their respective resulting cell state (i.e. 1 or 0).

In the case of $\phi_{k=1}$, each neighborhood *kernel* consists of components a, b and c yielding a cell state d each representing n possible values. In the binary CA case, there are two

such values 0 and 1. Such a $\phi_{k=1}$ could be generalized as:

$$\phi = [a, b, c] \rightarrow d \tag{1}$$

Each with 8 (n^{2k+1} or 2^{2k+1}) distinct combinations of a, b and c making up the set Q:

$$Q = \{a, b, c, ab, bc, ac, abc, 0\} \tag{2}$$

In total 256 ($n^{n^{2k+1}}$ or $2^{2^{2k+1}}$) possible subsets $R \subseteq Q$ and thus 256 λs that can be formed from ϕ_1 in the rule space. Thus a generic rule could be formed from $R_x \in Q$:

$$R_x \in Q = \{a, bc, ac\}$$

Whose $\lambda_{R_x \in Q}$ would be:

$$\lambda = f(x) = \begin{cases} 1 & x \in R_x \\ 0 & x \notin R_x \end{cases}$$

Applying the functions ϕ and λ iteratively, elementwise over a list of cells arranged in a 1-D array, treating out-of-bound elements as 0, the entire state-space of a binary ECA can be described. For instance, given the following initial state s_0:

$$s_0 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

And $\lambda = 00000010$ (or Rule 2) with only a single ϕ yielding an active bit of 1:

$$\phi = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \rightarrow 1$$

s_1 can be generated by iterating ϕ, over each component i of s_0 as:

$$s_1 = \begin{bmatrix} \phi(s_{0,0}) \\ \phi(s_{0,1}) \\ \phi(s_{0,2}) \\ \dots \\ \phi(s_{0,i}) \end{bmatrix}^T = \begin{bmatrix} \phi(\begin{bmatrix} s_{0,-1} & s_{0,0} & s_{1,1} \end{bmatrix}) \\ \phi(\begin{bmatrix} s_{0,0} & s_{0,1} & s_{1,2} \end{bmatrix}) \\ \phi(\begin{bmatrix} s_{0,1} & s_{0,2} & s_{1,3} \end{bmatrix}) \\ \dots \\ \phi(\begin{bmatrix} s_{0,i-1} & s_{0,i} & s_{1,i+1} \end{bmatrix}) \end{bmatrix}^T$$

Or:

$$s_1 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The entire state sequence can be represented by matrix $S = s_{0\dots t}$, where each row-vector is the state s_t at time step t and whose components i represent the state of each cell:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ s_{t,i} & s_{t,i+1} & s_{t,i+2} & s_{t,i+3} & s_{t,i+4} & s_{t,i+5} \end{bmatrix} = \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ \dots \\ s_t \end{bmatrix}$$

CARLA: a (novel) Cellular Automata Rule Learning Algorithm

After representing CAs in terms of ϕ, k and λ, learning cell update rules can be reduced to a search for a given configuration of discrete values for which *CARLA* can be applied. Using elements of Markov chains, the states of a CA can be processed using *CARLA* to find the λ parameter.

State Representation How the state space is represented has an impact on the computational overhead and required memory. For example, given the following states S:

$$S = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} \tag{3}$$

Each of the states of sequence S can be represented in either a top-down or bottom-up manner.

In the top-down approach, The state sequence could be represented using a *top-down* approach as states $S_i \rightarrow S_{i+1}$ or $\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \end{bmatrix}$.

This top-down approach incurs a high space complexity and memory overhead; the amount of information required to represent each state here is 2^n, where n is the number of bits required per cell.

Alternatively the *bottom-up* approach leverages the inherent properties of the cellular automata and represents the states in only the amount of memory required by the neighborhood of $\phi_{k=1}$, $2k + 1$ bits, instead of the entire state vector. This approach takes the *j*th neighborhood of $S_{i,(j-k)..(j+k)}$ as the input vector and the resulting state $S_{(i+1),j}$ as the output. Given the same sequence S (Eq. 3) and kernel ϕ (Eq. 1), for each bottom-up state s_i there is resulting sequence of transitions $t_{0\dots n}$ where n is the number of bits per state s. For transition $s_{1 \rightarrow 2}$ in S this could be illustrated as a matrix-vector pair where each row j corresponds to the kernel ϕ applied at cell j of s_1 and the j-th component in the column-vector would correspond to the value at cell $s_{2,j}$:

$$s_{i \rightarrow i+1} = \begin{bmatrix} \phi(s_{i,1}) \\ \phi(s_{i,2}) \\ \phi(s_{i,3}) \\ \dots \\ \phi(s_{i,j}) \end{bmatrix} \rightarrow \begin{bmatrix} s_{i+1,1} \\ s_{i+1,2} \\ s_{i+1,3} \\ \dots \\ s_{i+1,j} \end{bmatrix}$$

The bottom-up approach minimizes space complexity when representing the states. More akin to the structure of CAs themselves, the bottom-up approach is macro-state agnostic and local-first.

Transition Matrix Using this *bottom-up* state representation and building on the work of Dridi et al. (2019) demonstrating that state transitions of CAs can be at least partially modeled using Markov chains, a modified Markov transition matrix M^C (Table 1) keeps count of particular

$neighborhood \rightarrow state$ co-occurrences with minimal space complexity per Markovian state.

ϕ	$\rightarrow 0$	$\rightarrow 1$
$[1,1,1]$	62	2
$[1,1,0]$	98	6
$[1,0,1]$	19	1
$[1,0,0]$	27	105
$[0,1,1]$	7	31
$[0,1,0]$	5	70
$[0,0,1]$	2	41
$[0,0,0]$	50	5

Table 1: Example Transition Count Matrix M^C for a state sequence with an unknown λ

With the state model and M^C matrix, all the pieces are in place to derive λ_R from a given state sequence S. The resulting learning algorithm for learning the rule λ_R for a given ϕ $neighborhood$ set Q_ϕ associated with sequence of states S becomes:

function CARLA(S, Q_ϕ)
  $R \leftarrow \emptyset$
  M^C ▷ An empty transition matrix
  $t \leftarrow 0$ ▷ The lower threshold of counts
  for $s \in S$ **do**
    for 0 to i **do**
      for 0 to j **do**
        $M^C.increment(\phi(s_{i,j}), s_{i+1,j})$
      end for
    end for
  end for
  for $row \in M^C$ **do** ▷ Count frequencies
    if sum$(row) > t$ **then**
      if $row.colMax() = ActiveState$ **then**
        $R.add(row.key())$
      end if
    end if
  end for
  return λ_R

Validation Experiment

This algorithm can be tested by running it on sequences of space-time plots generated by each of the rules in a given rule space for a particular value of k, or each distinct neighborhood size. In this experiment, all 256 of the 8-bit, 1-D ECAs (Wolfram, 2002) rules are used to generate space-time plots over 32 consecutive time steps and running the *CARLA* algorithm on the resulting sequences. A rule is considered correctly learned if a sequence's learned rule matches the rule that was used to generate it or matches a rule forming a *twin sequence*, that is another rule of the same k producing an identical space-time plot given the same initial seed state. Using this process, all 256 rules are correctly learned.

The experiment also found that number of rules producing *twin sequences* was inversely proportional to the number of activated cells in the seed state s_0. As the number of activated cells in the initial seed approaches 50% of the total number of cells in the state s_t, the rate of rules that result in twin sequences begins to shrink as shown in Fig. 6.

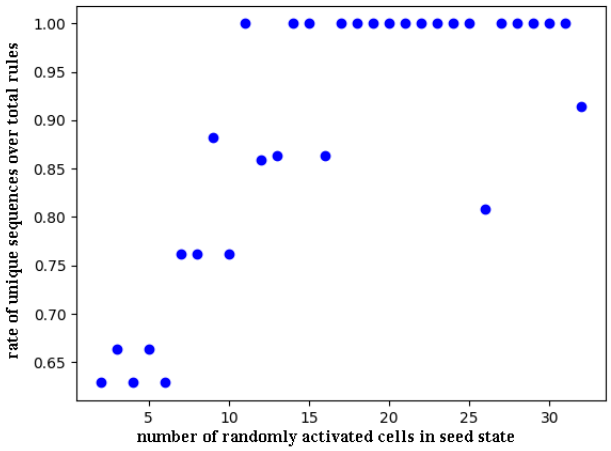


Figure 6: **Number of distinct sequences formed from seed states of varying numbers of activated cells** As the number of activated ("alive") bits per initial state S_0 increases the number of distinct sequences formed by each rule over all rules approaches 100% as greater numbers of bits are activated in the initial state.

A few exceptions seem to occur at 16, 26 and 32 activated cells specifically. This can be attributed to special configurations of cells such as "all alive" (i.e. 32 activated cells) or "checkerboard", in which every other cell is activated, resulting in high numbers of twin sequences. These results seem to be consistent with Gilpin (2019)'s results with CNNs, in which a relationship exists between the Shannon entropy of states and the performance of the learning algorithm.

Music Sequences as Cellular Automata

This section describes how the now-validated *CARLA* algorithm can be incorporated into a system for generating novel MIDI compositions.

Background: Artificial life and generative music

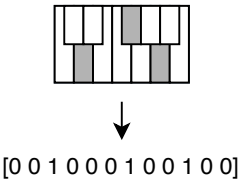
Music created by human experimental composers sometimes explores foundational themes in artificial life. In the 1930s and 1940s, movements such as *musique concrète* began to combine mechanical and biological processes into music by incorporating the recorded artifacts of living organisms and machines into musical works (Teruggi, 2015). By 1968, composers such as Steve Reich began connecting

music and its structure with processes exhibiting emergent, complex, and life-like dynamics such as pendulum motion.

Artificial life techniques have also been used directly to generate musical artifacts. By the end of the 20th century, evolutionary algorithms came to be used for generating new forms of *musique concrète* and cellular automata formed the basis of an entire subclass of computer music algorithms (McAlpine et al., 1999). Though perhaps not suitable for every generative use case, CA have a long history as components of generative computer music systems. MIDI files, which encode songs as sequences of discrete notes, in particular have proven to be a popular and intuitive domain for CA applications (Burraston and Edmonds, 2005). Typical approaches map CA states to MIDI parameters such as pitch and duration, essentially creating a sonic representation of a CA's evolution. Importantly, the CA rules in these cases are usually fixed *a priori*, and are either hand-coded (Dorin, 2002) or based on existing CAs such as Conway's Game of Life (Milen, 1990; Miranda, 2003). In contrast, the approach in this work (described in the next section) is to generate novel MIDIs using the *learned* CA update rules from processing note sequences in existing musical works.

Representing Music Using Space-Time Plots

Space-time plots can represent the state of the MIDI file as a sequence of arrays representing on and off states of a keyboard in a similar manner as CA active and inactive cells. For example, an array-form of a D-Major chord could be represented as in Fig. .



More generally, when all these states are tiled over a 128-component row vector representing note values and over n rows, one for each timestep, this is known as a pianoroll. The pianoroll can be thought of as a data structure similar to matrix S (Eq. 3). Each component of a pianoroll's row-vector consists of 128-bit integer values representing *velocity*, or how strongly the note is to be played by a MIDI instrument. For clarity and simplicity, we use only binary representations corresponding to on and off states of 1 and 0, respectively. However, higher fidelity representations can easily be generalized from binary.

Pianorolls may also be plotted using *time* and *pitch* on the x and y axes respectively. In Fig. 7, Rule 30 is shown with varying note duration assigned to certain pixels and each pitch value corresponding to the index of a bit in an 1-D automata space-time array.

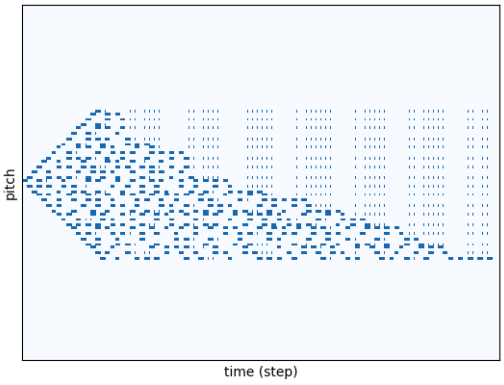


Figure 7: 8-bit Rule 30 as a piano roll. Each column is as a single state $s \in S$ starting with a seed state of 1 active bit.

To further reduce the state-space being represented, the pianoroll can be compressed into 12-unique notes with repeated states (i.e. long, legato notes held for multiple timesteps) collapsed into a single note and rests (i.e. states s without a single active bit) removed. This compressed music representation becomes a sequence matrix such as S:

$$s_{1\ldots n} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & \ldots & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & \ldots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & \ldots & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & \ldots & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & \ldots & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & \ldots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & \ldots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \ldots & 0 \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \\ s_{n,1} & s_{n,2} & s_{n,3} & \ldots & \ldots & \ldots & \ldots & \ldots & s_{n,j} \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ \ldots \\ s_1 \end{bmatrix}$$

Learning CA update rules of music sequences using *CARLA*

With music and cellular automata both represented as sequences of states over time using a space-time plot, a problem arises: if music sequences were to be generated using CAs, which rule should be selected to produce the *best* music? After all, even in the smallest case 8-bit rule space there are over 256 rules to choose from. In larger rule spaces formed by increasing the neighborhood radius k and enlarging the convolution kernel of ϕ, there are $n^{n^{2k+1}}$ distinct rules, where n is the number of possible states per cell. After taking just a 1-step increase to $k = 2$, the number of distinct rules baloons in orders of magnitude from 256 rules to 4,294,967,296 rules. Searching even the next-highest-bit rule space for the "best" rule thus becomes a challenge.

To solve the problem of searching the higher-bit rule spaces, *CARLA* can be used to instead *learn* the cell update

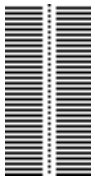


Figure 9: Space-time plot of $\lambda_{R_{4740813228047161360387}}$

rules of a state-sequence S made from MIDI files just the same as one made from CAs. The resulting rule can be interpreted as the *best matched rule*, or the rule describing a CA with state-change dynamics most similar to the piece of music provided.

Learning a Cell Update Rule from Chopin

By running the *CARLA* algorithm on a MIDI transcription of a solo piano composition by Chopin, known as *Waterfall* or *Études, Opus 10, No. 1* (Fig. 8), a *best matched rule* can be learned in the resulting rule spaces.

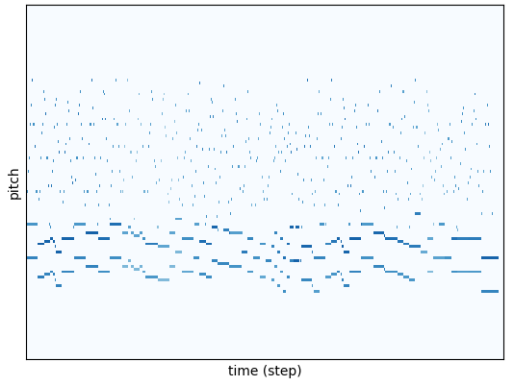


Figure 8: Chopin's *Études, Opus 10, No. 1* as a pianoroll

Although a trivial sequence can be formed using a $k = 1$ and discovering one of the 256 8-bit rules, by using increasingly larger k radii in each provided ϕ *neighborhood*, higher fidelity rules from larger rule spaces can be learned to find a λ parameter that best matches the M^C of *Waterfall*'s state sequences. After a few attempts made using a kernel of $k = 1$ for ϕ function and then $k = 2$, the *best matched rule* that produces the best sounding result for *Waterfall* grew from a ϕ convolution kernel neighborhood of size 7 ($2k + 1$ given $k = 3$).

Initially, the resulting cell update rule learned by $CARLA(\textit{Waterfall}, \phi_{k=2})$ is the 128-bit rule $\lambda_{R_{4740813228047161360387}}$ whose space-time plot (Fig. 9) is particularly dense and zebra-striped.

The zebra-striping occurs in the first-half of all odd numbered rules. This follows intuition given that odd-numbered rules in all rule-spaces have a 0 in their first bit and 1 in their last bit using binary representation:

00000000000000000 0000000000000000
0000000000000000 0000000100000001
0000000000000000 0000000100000001
0000000000000000 000000000000001**1**

Consequently the following transitions are present in each of their λ_R sets:

$$[0, 0, 0, 0, 0, 0, 0] \rightarrow 1 \tag{4}$$

$$[1, 1, 1, 1, 1, 1, 1] \rightarrow 0 \tag{5}$$

The first transition can be described as the spontaneous activation of a single cell in state s_t given a fully inactive neighborhood in the previous state s_{t-1} and the second its exact opposite, which transitions neighborhoods of all activated cells into in inactive states. When using a particularly sparse initial seed state, these two transitions described in Eq. 4 and Eq. 5 result in the zebra pattern seen above. Thus, for the purposes of generating a sequence of music from learned CA rules, a system of post-processing ought to be applied to improve the quality of the resulting space-time plots after *CARLA* prunes the rule space. One possible approach is to use binary arithmetic to set the first bit to 1 and the last bit to 0 in the binary representation of each learned rule to avoid problematic states. For example, the aforementioned rule could be easily transformed to an even-numbered rule by subtracting 1 and becoming $\lambda_{R_{4740813228047161360386}}$, immediately resulting in the following space-time plot (Fig. 10):

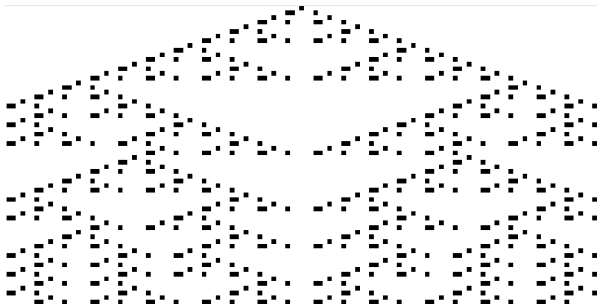


Figure 10: Space-time diagram of $\lambda_{R_{4740813228047161360386}}$

This single post-processing step results in a more sparse space-time plot that, at least when rendered into standard music notation, visually passes as music (Fig. 11).

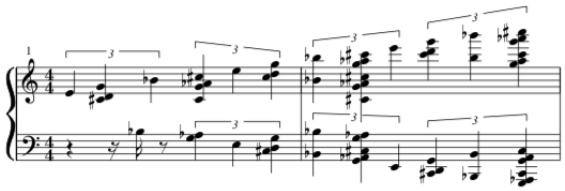


Figure 11: A piano score generated using a space-time sequence for $\lambda_{R_{4740813228047161360386}}$

Refining the Search for The Best Update Rules

As post-processing is introduced, the space-time plot begins to resemble conventional music score, albeit with unrealistically high density. This high density of activated cells conflicts with the sparse nature of conventional music in form because music generally consists of a finite, low number of simultaneous voices performed at once, or *polyphony*. In the case of piano scores designed for human hands of five fingers each, polyphony does not surpass 10 and Fig. 11 rapidly exceeds that, so it fails to resemble the aforementioned sparsity of the original *Waterfall* piece and most anthropocentric piano scores. A way of controlling the resulting polyphony is crucial for generating music for humans.

We use a novel space-time plot postprocessing algorithm called *Tendril*[1] to produce higher sparsity and add controls for plausible polyphonic qualities. In the spirit of biomimicry, *Tendril* imitates a tendril in the natural world wrapping itself around a trellis or tree. A 1-cell *tendril* can be grown over the space-time diagram in order to reduce the polyphony to 1 bit for each time step for each pass of the algorithm, shown here in Fig. 12.

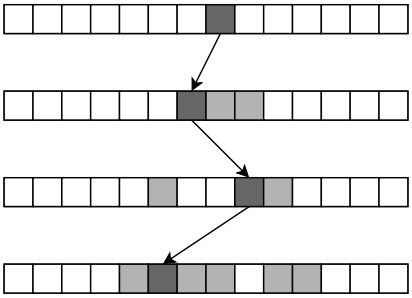


Figure 12: A single-pass *Tendril* being applied to a *space-time* diagram of Rule 30. Darker cells are sampled by the algorithm from each state, lighter cells ignored.

Many selection strategies can be designed for *Tendril*, such as choosing a random cell at each time step or applying a random walk. A random walk, for example, could stochastically choose a "left" or "right" movement as it moves rowwise across the CA states S, taking 1 active cell from each state s_t at each time step t closest to its current cell in the chosen direction. This automatically reduces the *polyphony* of each state to 1 active bit for each $s_t \in S$.

Repeating this process v times, once per desired voice of polyphony and mapped to varying MIDI time intervals, a piece of comparable *polyphony* emerges, much more similar in note density to its source *Waterfall*. Thus, given the following *space-time diagram* built using the cell update rule *CARLA* learned from *Waterfall* then mapped to the C minor scale:

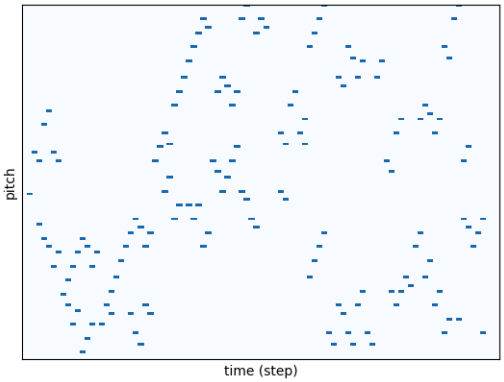


Figure 13: A pianoroll for a piece generated using *Tendril* from $\lambda_{R_{4740813228047161360386}}$'s space-time plot and mapped to C minor scale

When mapped to the C minor scale for each desired voice of polyphony, a 2-voice, polyphonic sequence emerges. A 1-voice pattern results generated by post-processing the resulting *space-time diagram* with a random-walk-based *tendril* :

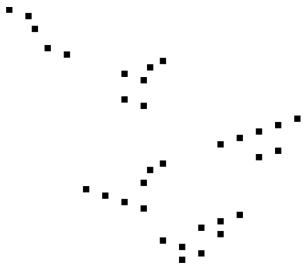


Figure 14: A "tendril" or 1-bit per-state sampling of the space-time diagram previously shown in Fig. 10

Thus, using a simple iteration of the *Tendril* algorithm for each desired voice or track in the output sequence, an entire MIDI file can be grown from a single automata rule learned in an unsupervised manner by *CARLA* and pruned using *Tendril*. A demonstration recording of a multi-pass *Tendril* run, once for $v = 2$ at 60bpm with

[1] https://github.com/omardelarosa/tendril.

each cell given an *eighth-note* duration and then again for $v = 1$ with each note given a *quarter-note* duration, can be heard at `https://soundcloud.com/ioximusic/tendrill-in-c-minor`.

Discussion and Conclusion

The previous sections describe two possible approaches for learning cell update rules from sequences of states and generating music files from CAs with low space complexity. While the learned cell update rules focused on 1-D CAs and the music sequences focused on pitch selection, much more could be explored in higher-dimension CAs and on properties of music beyond pitch selection.

The *Tendril* algorithm for sequence generation selected *pitch* and used only static *rhythm* representation giving each note the same duration for conceptual clarity. However, pitch need not be the only property of music sequences controlled by CA cell update rules. The work of Brown (2005) explores how generative music systems focusing on *rhythm* could be designed using 2-D CAs.

Going beyond learning the cell update rules of 1-D CAs, CARLA generalizes to higher dimensional CAs such as 2-D or 3-D with minimal hyperparameter adjustments. It is also conceivable that 2-D cell update rules could be learned using CARLA and image-based, abstract visual pattern generative systems could be made using *Tendril*. However, objectively measuring the quality of creative artifacts proves difficult and a simple measurement for 2-D visual pattern quality is beyond the scope of this paper.

This paper demonstrates that a method of learning cellular automata update rules and generating new sequences from those update rules can be made to match the properties of CAs themselves. The learning algorithm *CARLA* provides a low-space-complexity method of learning the cell update rules with a comparable degree of accuracy on par with CNN systems. By using *CARLA* on non-CA data such as MIDI music files to learn their underlying cell update rules and post-processing using the biomimetic *Tendril* algorithm, a 2-stage generative system for growing new musical sequences from CA cell update rules emerges.

Acknowledgements

This work was supported by the National Science Foundation (Award number 1717324 - "RI: Small: General Intelligence through Algorithm Invention and Selection").

References

Andrew R. Brown. 2005. Exploring Rhythmic Automata. In *Applications of Evolutionary Computing*, Franz Rothlauf, Jürgen Branke, Stefano Cagnoni, David Wolfe Corne, Rolf Drechsler, Yaochu Jin, Penousal Machado, Elena Marchiori, Juan Romero, George D. Smith, and Giovanni Squillero (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 551–556.

Dave Burraston and Ernest Edmonds. 2005. Cellular Automata in Generative Electronic Music and Sonic Art: A Historical and Technical Review. *Digital Creativity* 16, 3 (2005), 165–185.

Alan Dorin. 2002. LiquiPrism: Generating Polyrhythms with Cellular Automata. In *Proceedings of the 2002 International Conference on Auditory Display*. International Community for Auditory Display, 447–451.

Sara Dridi, Franco Bagnoli, and Samira El Yacoubi. 2019. Markov Chains Approach for Regional Controllability of Deterministic Cellular Automata, via Boundary Actions. *Journal of Cellular Automata* 14, 5/6 (2019), 479 – 498.

William Gilpin. 2019. Cellular Automata as Convolutional Neural Networks. *Phys. Rev. E* 100 (Sep 2019), 032402. Issue 3.

Y. Jing, Y. Yang, Z. Feng, J. Ye, Y. Yu, and M. Song. 2019. Neural Style Transfer: A Review. *IEEE Transactions on Visualization and Computer Graphics (early access)* (2019).

Kenneth McAlpine, Eduardo Miranda, and Stuart Hoggar. 1999. Making Music with Algorithms: A Case-Study System. *Computer Music Journal* 23, 2 (1999), 19–30.

David Milen. 1990. Cellular Automata Music. In *Proceedings of the 1990 International Computer Music Conference*. ICMA, 314–316.

Eduardo Reck Miranda. 2003. On the Evolution of Music in a Society of Self-Taught Digital Creatures. *Digital Creativity* 14 (2003), 29–42. Issue 1.

Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. 2020. Growing Neural Cellular Automata. *Distill* (2020). `https://doi.org/10.23915/distill.00023` https://distill.pub/2020/growing-ca.

John Von Neumann and Arthur W. Burks. 1966. *Theory of Self-Reproducing Automata*. University of Illinois Press, USA.

Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 3645–3650.

Daniel Teruggi. 2015. Musique Concrète Today: Its reach, evolution of concepts and role in musical thought. *Organised Sound* 20, 1 (2015), 51–59.

Stephen Wolfram. 2002. *A New Kind of Science*. Wolfram Media.